



Test and Reliability

Chapter 1: Introduction to Circuit Testing

Prof. Dr. Ralf Wimmer
Dr. Tobias Seufert

Summer Term 2026



What is Testing?

- Manufacturing processes of VLSI chips may introduce different kinds of **defects** (shorts, opens, etc.).
 - No 100% yield possible.
- Failures may also occur **during the lifetime** of a circuit.
- As a result, a circuit may be different from its design.
- The **goal of testing** is to identify the existence of such a difference.

Formal verification *in contrast*: prove that circuit has been *designed* correctly. Related to, but different from testing.

Effects of Defects

- Some defects may be **catastrophic**—they will be detected as soon as we try to use the circuit.
- Other defects may be **subtle** and may require very specific tests in order to be detected.
- **Latent** defects do not cause any direct malfunction, but they deteriorate over time
- **Transient** and **intermittent** faults come and disappear
 - Transient faults not repeatable, but, e.g., caused by radiation
 - Intermittent fault activated (by some unknown activation condition)

Why Test?

- Test is expensive: up to 60% of overall manufacturing cost
 - exact cost depends on quality requirements
 - no 100% guarantee!
- Selling a defective part is expensive, too!
 - a defective IC invalidates whole board / product
 - 'Rule of ten': cost of fixing increases roughly by 10 from component → board → system → field
- **Whether** to test and **how well** to test depends on which cost is higher
 - safety-critical systems: affect human life
 - 'non-replacable hardware' for use in space (e. g., satellites)

Outline: Introduction

- Defects, faults, coverage
- Test application
 - ATE, BIST, Test compression, DFT
- Fault models
 - Behavioral; Functional; Structural; Switch-level
- Stuck-at faults
 - detection, redundancy, equivalence, dominance
 - fault collapsing

Fault Coverage and Yield

- The process **yield** (Y)
 - The fraction of manufactured parts that are defect-free
- **Defect level** (DL)
 - The fraction of bad parts that pass all the tests
 - usually specified in DPM (defects per million), sometimes called PPM (defective parts per million)
 - often the main target of test development
- **Fault coverage** (FC)
 - measure of the quality of a test
 - used to control defect level

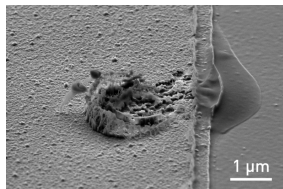
Note: Defects / Faults

- **Defects** are actual *physical effects* that happen in a manufactured circuit.
- **Faults** are used to *model defects*.
- Not every defect may be modeled by a fault. Not every fault may correspond to a likely defect.
- However, faults are an effective tool since it is impossible to enumerate all the possible defects and determine their effect on the circuit.

Systematic Defects

Systematic Defects: Can be addressed by improving the manufacturing process.

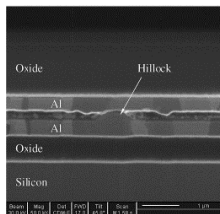
Corrosion



- e.g., by contamination with chlorine from etching
- shorts and opens
- Fix: passivation and resist cleaning after each step

Source: nanofab.ualberta.ca

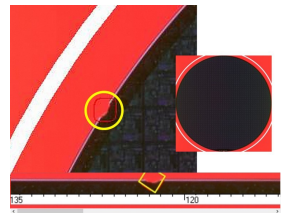
Al hillocks



- Different rates of thermal expansion between Al and Si
- Fix: Dope Al with Cu, cap Al with W / Ti, ...

Source: Lin, Stevenson, Gundlach,
Dunare, Walton

Edge defects



- Particles from machines that hold wafer; can get to other parts of wafer
- Fix: etch aggressively, use thicker oxide film

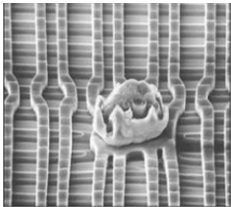
Source: microtronic.com

Random Defects

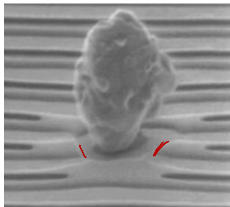
Random Defects: Can be reduced, but not eliminated completely.

- Particles

Short



Open



- Etch chambers, wafer transport
- Fix: reduce particle levels; add cleaning

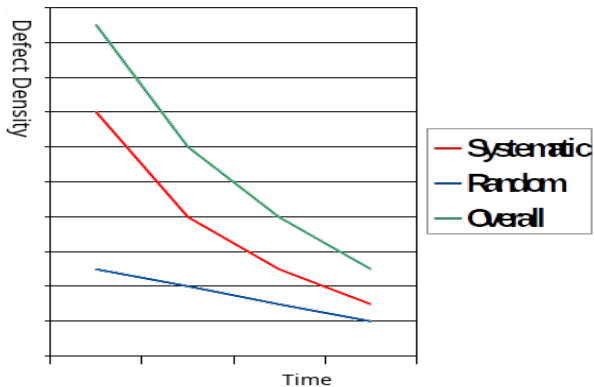
- Mask defects
 - Random (particle induced)
 - Result in 0% yield
- Interlevel dielectric pinholes
 - Overetching, hillocks, particles
 - Can result in a short defect or trap moisture and become a reliability issue
- Scratches
 - Wafer handling within equipment or by people
 - Cause opens; may contaminate neighboring locations

Fault Coverage, Defect Level and Yield

- M : number of modeled faults.
- N : number of modeled faults detected by a test.
- Fault Coverage: $FC = N/M$.
- Assuming that all the faults are *equally likely* and *independent*, it can be shown that Defect Level $DL = 1 - Y^{(1-FC)}$
 - No testing required if $Y = 100\%$! It would be $DL = 0$.
- Improved formulas exist.

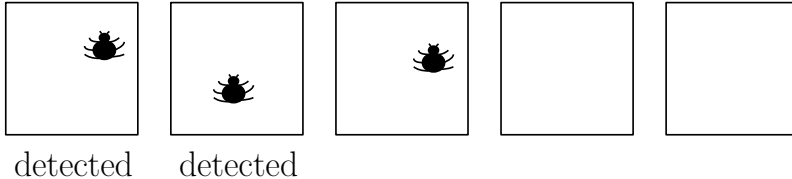
Yield in Time

First silicon: Yield approx. 20–30%.
Production: Yield approx. 80–95%.



- Yield improves over time with decreasing defect density
- Yield never becomes 100%
- When yield is close to 100%, equipment may be used for next-generation technology (with low yield, again)

Example: Actual Parts



- We will ship one bad part, defect level: 'one out of three'
- The defect level represents the general case
- We are interested in generating tests to maximize Fault Coverage

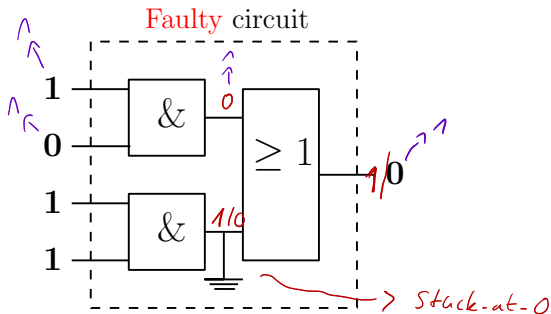
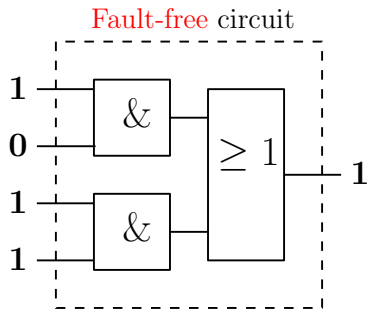
Introduction: Types of Test



Types of Tests — Output

- All types of tests **specify input values** and **measure output values**.
- Logical tests:
 - Apply logical values on circuit inputs. Consider logical values on circuit outputs (0,1 values).
 - voltage levels: Logic-0 $\approx 0V$, Logic-1 $\approx V_{DD}$
 - For defects that cause a change in the logical behavior of the circuit (an output value is 1 when it is supposed to be 0).

Types of Tests — Logical Test *Example*



—00, —01, —10, 11—
are not tests for this fault

Logical Test — *Fault-injected circuit*

```
module fault_free (  
    input wire A,B,C,D,  
    output wire Y  
);  
wire AND1_out, AND2_out;  
assign AND1_out = A & B;  
assign AND2_out = C & D;  
assign Y = AND1_out | AND2_out;  
endmodule
```

Fault-free design represented using an HDL

```
module faulty (  
    input wire A,B,C,D,  
    output wire Y  
);  
wire AND1_out, AND2_out, faulty_AND2_out;  
assign AND1_out = A & B;  
assign AND2_out = C & D;  
assign faulty_AND2_out = 1'b0;  
assign Y = AND1_out | faulty_AND2_out;  
endmodule
```

Fabricated circuit. Modeled by the fault-free design with a fault injected into it; represented using the same language

Types of Tests — Electrical Tests

- Measure the values of electrical parameters such as **voltage** levels (most often), **current** levels, or **delays**.
 - I_{DDQ} testing:
 - applied to CMOS circuits.
 - measures the quiescent power supply **current** drawn when the chip is not switching ('leak current').
 - in a fault free chip this current is very small. In the presence of a defect such as a short, the current may be significantly higher.
- There are other tests than electrical
 - e.g., optical inspection, thermal maps

I_{DDQ}
Supply
current

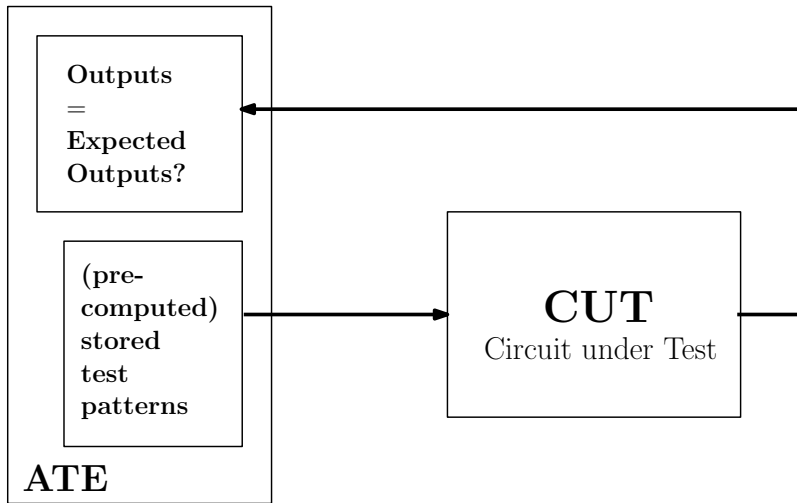
Types of Tests — Use

- **Detection**
 - Distinguish between good and bad circuits. Needed in order to avoid shipping or using bad circuits. (What we have just seen)
- **Diagnosis**
 - Identify the **location** of the defect in the circuit. Needed in order to improve the manufacturing process or for repair.

Types of Tests — Test Application

- External test:
 - The input and output values are stored in an external tester, so-called **Automatic Test Equipment (ATE)**.
 - The external tester applies the input values and compares the output values to the stored values.
 - Requires an external test generation process.
 - ATE are expensive. Cost increases with the amount of memory required and the maximum supported clock frequency.

Automatic Test Equipment (ATE)



Automatic Test Equipment (ATE) — Examples



Figure 1: Advantest T5592,source: semicondutoronline.com



Figure 3: Teseda WorkBench,source: eetimes.com



Figure 2: NPTest / Credence EXA3000,source: wavecrestkk.co.jp

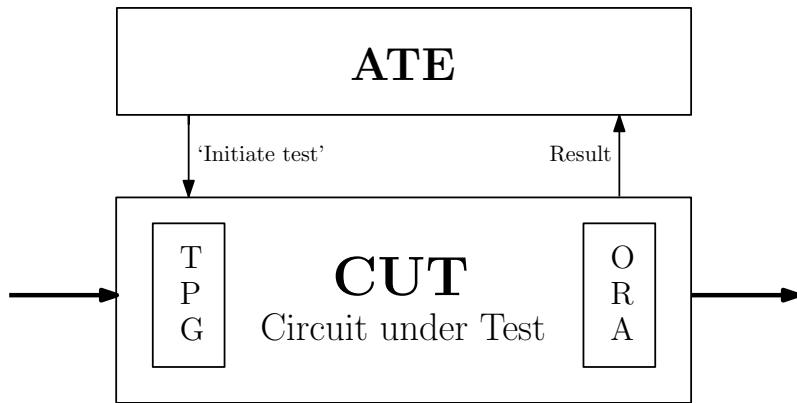


Figure 4: Advantest T6391,source: advantest.com

Built-In Self-Test (BIST)

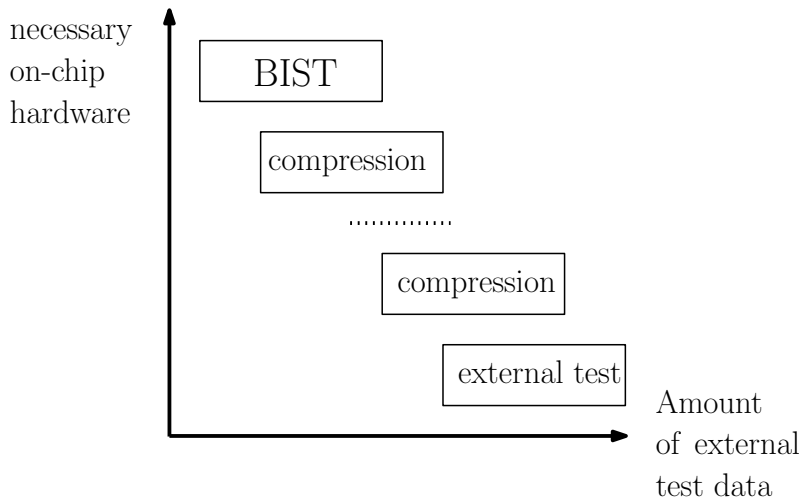
- The input values are generated by special hardware **on-chip**, e.g., a linear feedback shift register (pseudo-random number generator)
- Output values are compressed on-chip into a **signature**.
- The external tester sends a signal to start the testing process, and receives the signature at the end.
- It compares the signature to a fault-free signature to determine whether the circuit passed the test.

BIST — Schematic View



TPG—Test Pattern Generator; ORA—Output Response Analyzer

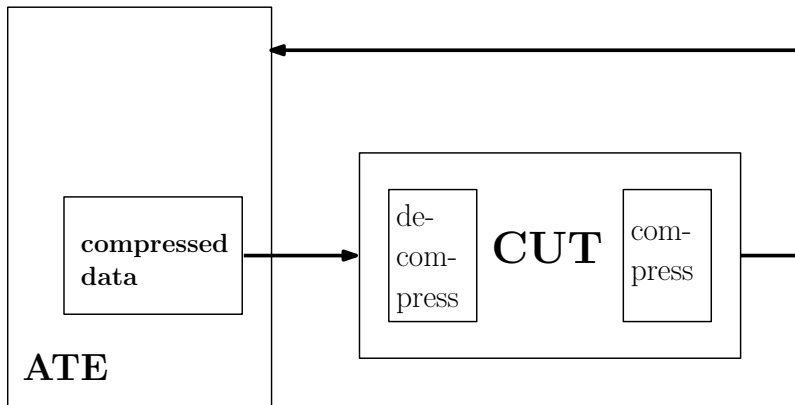
Illustration: Test Data Compression



Test Data Compression

- Input values are stored in an external tester in a **compressed form**.
 - Reduces the memory requirements of the ATE and therefore its cost.
 - Reduces time for test, energy, stress of the circuit.
 - Fewer computations required on the ATE-chips often have to be tested at full speed!
- The circuit contains a decompression circuit that applies the required tests to the circuit.
- Output compression can be done similar to BIST.

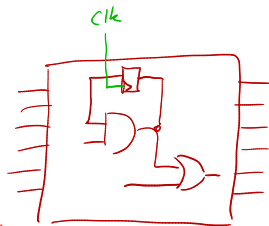
Illustration: Test Data Compression



Design-for-Testability (DFT) (Synthesis-for-Testability)

- Methods of designing or augmenting a circuit to reduce the cost of testing it.
- The cost of testing is estimated at up to 60% of the cost of manufacturing a VLSI chip.
- Cost is determined by:
 - the cost of test generation.
 - tester cost determined by test set size and test application time.

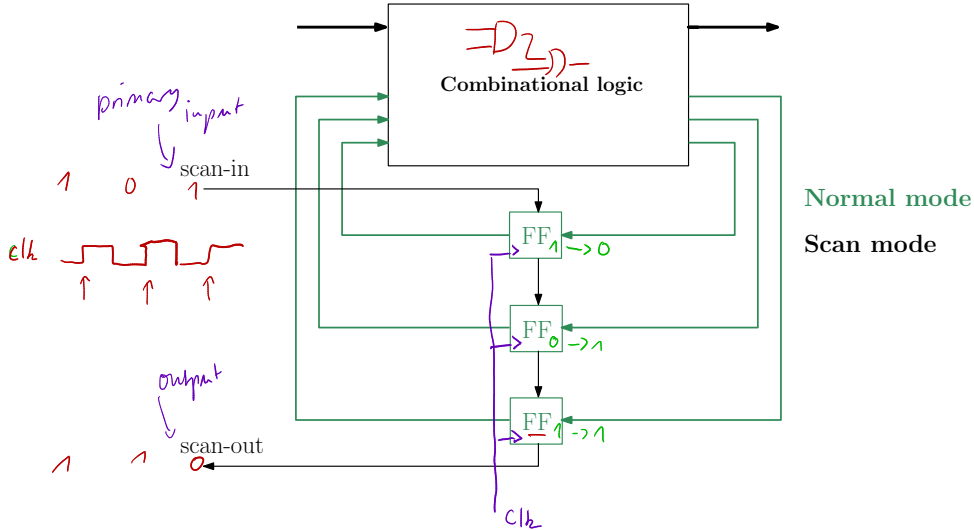
Circuit Types



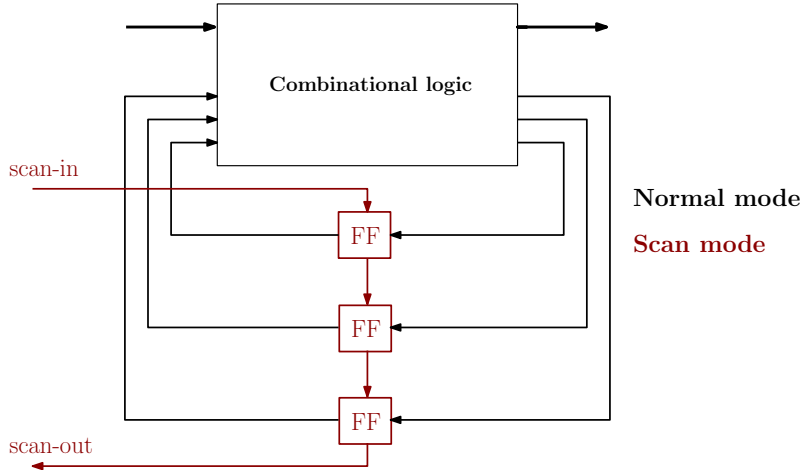
- Digital
 - combinational
 - synchronous Sequential (without Scan)
 - synchronous Sequential with Scan
 - asynchronous
- Analog
- Mixed Signal
- Memories; MEMS; Sensors; ...

We focus on **synchronous digital** circuits

Design-for-Testability — Scan Chains



Design-for-Testability — Scan Chains



Design-for-Testability — Scan Chains

- If *scan mode* is activated
 - The scan chain can be used to assign values to the state variables for every vector applied to the circuit.
- If *normal mode* is activated
 - Find a test for the combinational logic of the circuit. Suppose the FFs need to have the values (111).
 - Bring the circuit from its initial state to state (111) using a **transfer** or a **synchronizing sequence**
 - ⇒ length of synchronizing sequence might be exponential in size of memory!
 - Apply an appropriate primary input pattern.

Observing the Fault Effect

- If the fault is propagated to a **primary output**, the previous process is sufficient.
- Otherwise:
 - The scan chain can be used to assign values to the state variables for every vector applied to the circuit.
- If *normal mode* is activated
 - One or more flipflops store a wrong value (fault effect).
 - In the **non-scan case**, add a sequence that will propagate the fault effect to a primary output.
 - In the **scan case**, use the scan-out line to read the flipflop values.

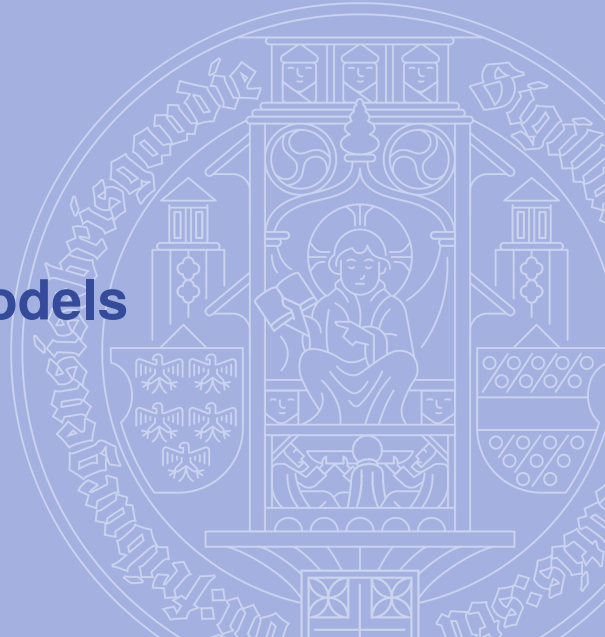
Who Needs to Know About Test?

- Test engineers, but also designers, including senior designers, and managers!
- Test up to 60% in cost
- Test issues have to do with both design and manufacturing!
 - In many cases, manufacturing is done at an other location or even by a different company
 - Test development sometimes outsourced completely
 - 100% automation is a miracle

Test Issues in a Typical Project

- Project Manager
 - Responsible for performance, quality (DPM), yield, area, power consumption, cost, development time trade-off
- Testability Manager
 - Selects design for manufacturability rules (area/yield/DPM), DFT (area/tool cost/time/DPM), tools to be used (cost/time), tester hardware, ...
- Designer (senior)
 - Decides on DFT strategy, given the targets
- Design-for-Testability (DFT) engineer
 - Designs DFT hardware
- Test engineer (simulation)
 - Runs fault simulation and ATPG; generates data for DFT
- Test engineer (ATE)
 - Programs tester, prepares probecard, fine-adjusts timing ...

Introduction: Fault Models



Functional vs. Structural Approach

- **Functional**: check whether functionality meets specification
-
- What is 'specification'?
 - combinational circuit: essentially truth table
 - sequential circuit: Mealy automaton (= transition + output function)
 - **Infeasible** for non-trivial ICs
 - In practice: check **the 'relevant' parts** of functionality, typically using hand-written tests based on designer's experience

Functional vs. Structural Approach

- **Structural**: prove absence of defects
-
- **Model** defects as faults
 - **Generate tests** that detect (a large fraction of) faults
 - can be automated
 - The quality of functional tests can be assessed using structural methods
 - empirically better than structural tests with same quality assessment (fault coverage)

We focus on structural methods

Fault Models: Overview

- Fault models are used for representing **defects at higher levels of abstraction**.
- They are important in making the test generation problem **tractable**.
 - A higher level of circuit description contains **fewer details** and it is therefore **easier to manage**.
 - The number of faults defined at a higher level is **smaller** and their effect on circuit behavior is **easier to model**.
- They are based on the (empirical) observation that tests for fault models at a higher level are effective in detecting faults at a lower level, and are effective in detecting defects.

Circuit Description Levels

behavioral level

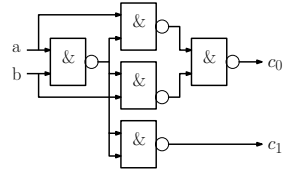
```
c = a + b;
```

⇒ functional level



```
wire a, b;  
wire [1:0] c;  
adder (c, a, b);
```

⇒ structural level



```
module adder(c, a, b);  
input a, b;  
output [1:0] c;  
wire d,e,f,g;  
nand (d, a, b);  
nand (e, a, d);  
nand (f, d, b);  
nand (c[0], e, f);  
nand (c[1], d, d);  
endmodule
```

Circuit Description Levels

- **Behavioral**
 - Describes data and control flow using a hardware description language (VHDL, Verilog).
- **Functional**
 - Represents the circuit using registers, modules and interconnect structures (RTL).
- **Structural**
 - Represents the circuit at the logic level using gates.



(lower levels)

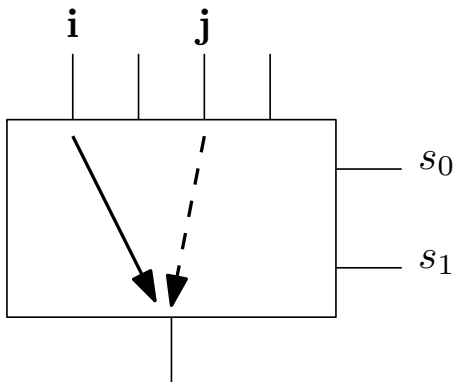
- **Switch-Level**
 - A transistor-level description of the circuit.
- **Geometric Description**
 - layout level description.

Behavioral Fault Models

- Suppose that 1 and 0 are the highest and the lowest values of the value system, respectively.
- A variable R is always at 1 or 0.
- A function call always returns 1 or 0.
- A statement ' $\text{if}(Y)\text{then}\{B_1\}\text{else}\{B_2\}$ ' might be erroneous and
 - always executes B_1 ,
 - always executes B_2 , or
 - executes B_1 when Y is *false* and B_2 when Y is *true*.
- ...

Functional Fault Models

- Ensure that a functional block implements the function it was designed for. Example: Multiplexer (MUX)
 - When an input i is selected, another input j is selected instead of or in addition to i .



Truth Table Based Models

- The truth table of the functional block can change in an arbitrary way.
- Requires **exhaustive testing** of the block. A block with n inputs requires 2^n tests.
- The number of tests can be reduced if output i depends on n_i inputs only. Output i can be tested using 2^{n_i} tests (pseudo- exhaustive testing).

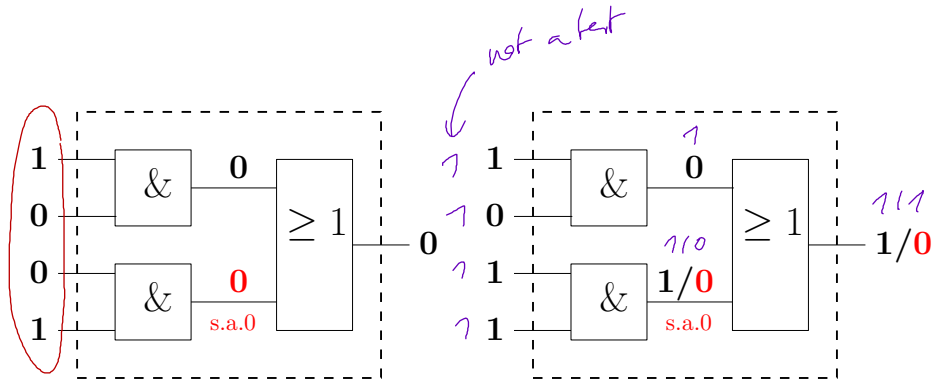
Structural Fault Models



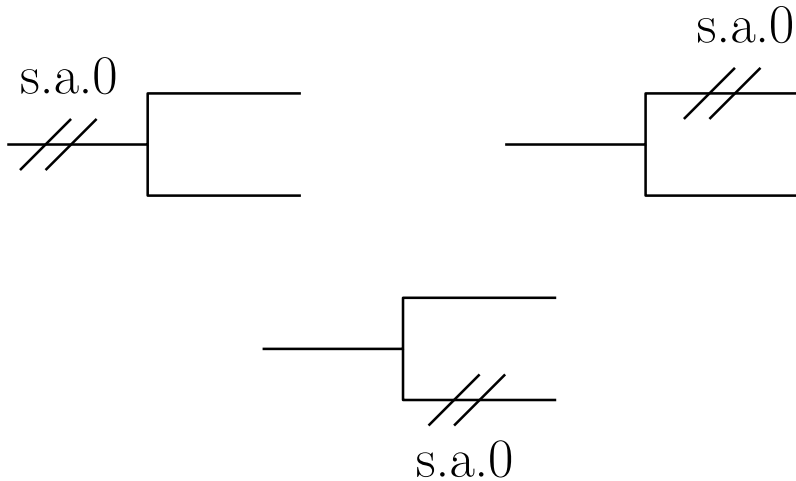
- The **stuck-at fault model**:
 - A line is stuck-at 0 if it remains **fixed** at a low voltage level.
 - A line is stuck-at 1 if it remains **fixed** at a high voltage level.
- Represents **shorts** to the power line or ground, as well as other shorts and opens.

This is the most commonly used fault model!

Single Stuck-At Fault



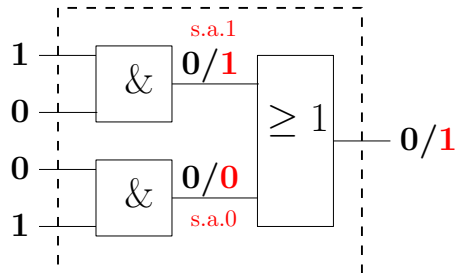
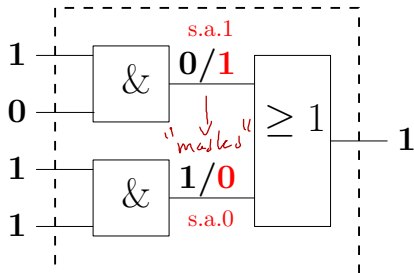
Fanout Stems and Branches



Structural Fault Models

- Under the **single stuck-at fault** model, faults are assumed to occur on single lines in the circuit.
- Under the **multiple stuck-at fault** model, faults may occur on more than one line simultaneously.
- For a circuit with L lines, there are $2L$ single stuck-at faults, and $3^L - 1$ multiple stuck-at faults.
 - $3^L - 1$, why? Choose between 'no fault', 's.a.0', 's.a.1' for each line (all combinations!). Minus one case—namely the fault-free case, no 'multiple-stuck-at-fault' to consider.

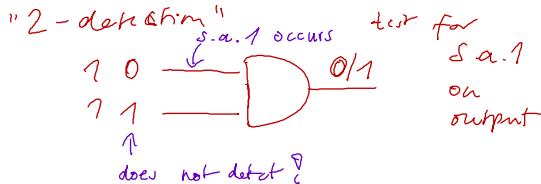
Multiple Stuck-At Fault



Justification for Considering Single Faults

- For periodic testing that is done often enough, one can assume that if a fault occurs, testing will take place before another one occurs.
- Tests for single stuck-at faults **prove effective** in detecting multiple stuck-at faults as well as various types of defects.
- **Easier to handle** than multiple faults.
- **Not accurate** for actual defect mechanisms!
 - defect clustering
 - an issue when performing diagnosis

Extension — n -Detection Model

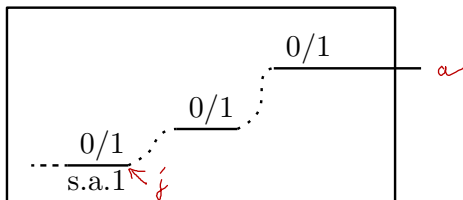


- Detect each stuck-at fault $n > 1$ times, by n different tests.
- Enhances the ability of the test set to detect defects.
- n can be fault-independent, or fault-dependent (variable n -detection).

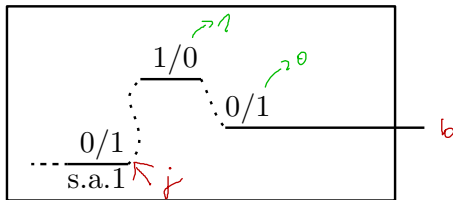
n -Detection Test Set

test for s-a.1
on line j'

test 1



test 2



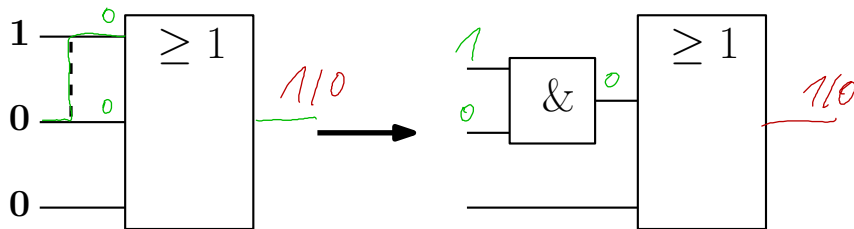
Bridging Faults



- A **bridging fault** is an unwanted connection between two lines in the circuit.
- Assuming that the resistance of the connection is zero simplifies the modeling.
- Furthermore, a bridging fault can be modeled by a **wired-AND** or a **wired-OR** between shorted lines.
- This is not always accurate.
- Bridging faults are best modeled at the **layout level** (geometric description).

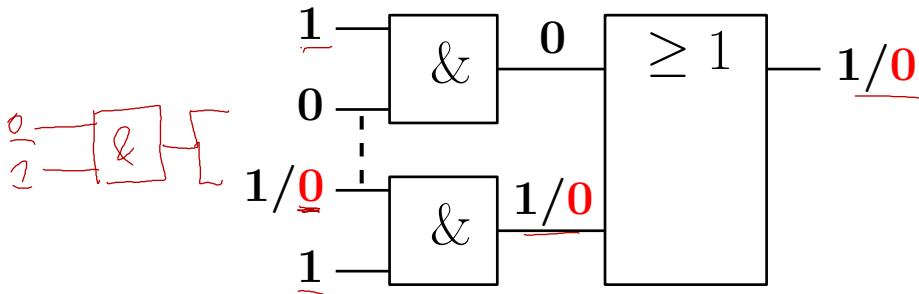
Bridging Faults—‘Wired-AND’

a test:



Side remark: A wired-OR would not be detectable in this case.

Bridging Faults—Another ‘Wired-AND’

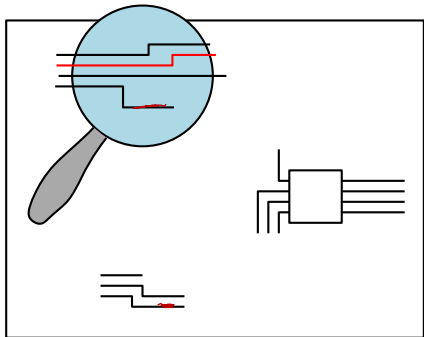


Wired Bridging Faults

- **AND** corresponds to **0** being the stronger value.
- **OR** corresponds to **1** being the stronger value.
- It is also possible to associate strength with a line, not with a value:
 - when g_1 is bridged with g_2 , g_1 (or g_2) is stronger.
- **The 4-way model**: Four faults per pair of lines (g_1, g_2) depending on which one is stronger, and for what value.

The Number of Bridging Faults

- For a circuit with L lines, $L \cdot (L - 1)$.
- The number can be **restricted** using **layout information** to find k lines that are most likely to be bridged with each line. Total $k \cdot L$ faults.



Delay Faults

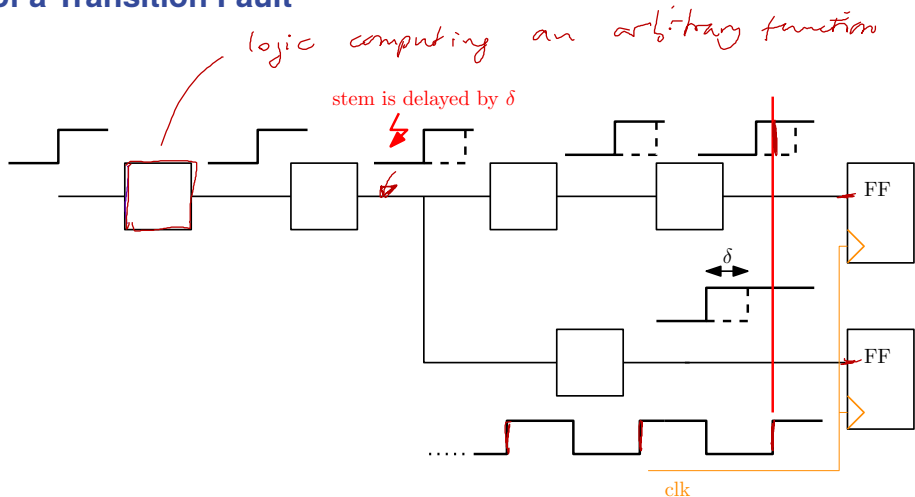
- Model defects that affect the **timing behavior** of the circuit.
- Typically, defects that cause the circuit to be **slower than required** by its designated speed of operation.
- Delay defects affect the propagation delays of signals in the circuit.

Delay Fault Models



- **Transition faults** (gross delay faults)—the propagation delays through one or more lines are **higher than the circuit clock period**.
 - a **slow-to-rise** fault slows the **rising** transition on a line.
 - a **slow-to-fall** fault slows the **falling** transition on a line.

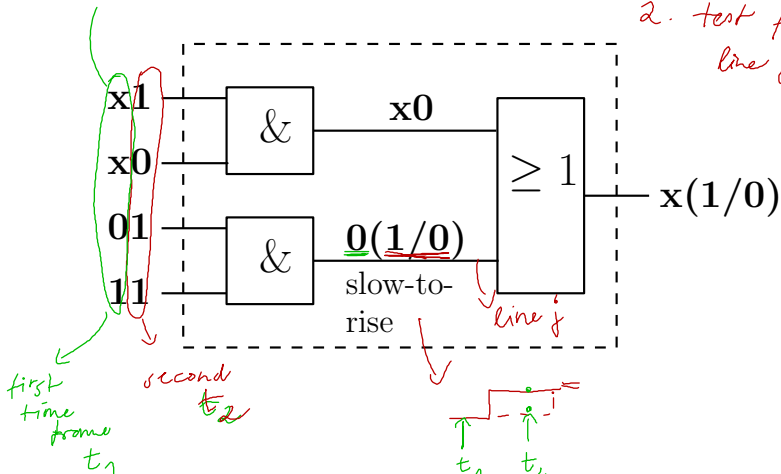
Effect of a Transition Fault



Testing for a Transition Fault

X : don't care ! " _ "

1. produce a 0 on line j
2. test for s.a.-0 on line j



Relationship between Transition Faults and Stuck-at Tests

How to test for a *transition fault* $g : a \rightarrow \neg a$?

- The **first pattern** of the test needs to set $g = a$. Weaker requirement than detecting g stuck-at $\neg a$.
- The **second pattern** of the test needs to set $g = \neg a$. The fault effect will be $g = \neg a/a$.
- To detect the fault it is necessary to propagate $g = \neg a/a$ to an output.
- This requires a test for g stuck-at a .

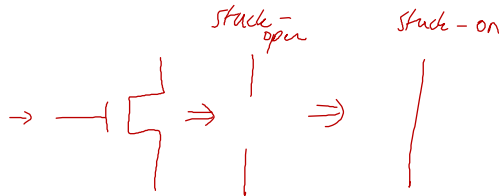
Gate Delay Faults

- For one or more gates, the **propagation delay** from an input to an output is **larger than designed** for.
 - slow-to-rise
 - slow-to-fall
- A gate delay fault may be detected through a **long path**, but not through a short path, depending on the size of the delay.

Path Delay Faults

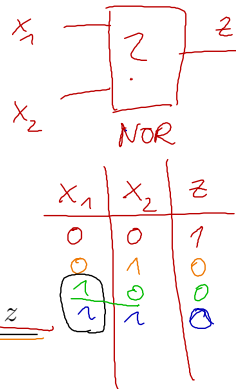
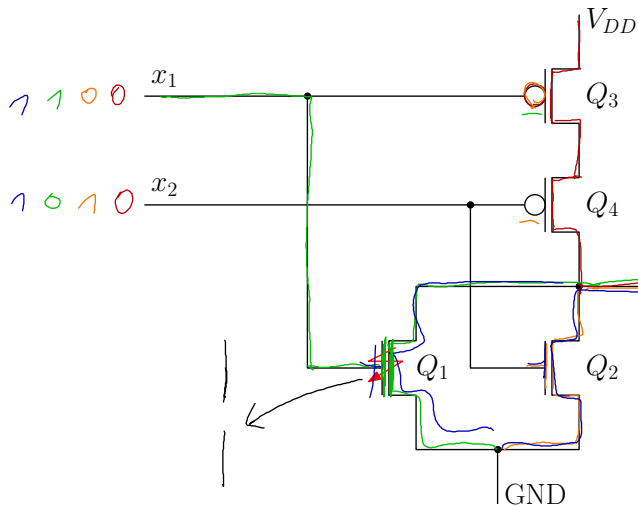
- One or more paths from inputs to outputs are slow to propagate a rising or a falling transition.
- Models **cumulative effects** of delay variations of gates and lines along the path.
- All the delay faults require **two-pattern tests**.

Switch-Level Fault Models



- Deal with transistors in CMOS circuits.
- Under a **stuck-open** fault, a transistor is **permanently non- conducting**.
- Under a **stuck-on** fault, a transistor is **permanently conducting**.

A Stuck-Open Fault



$\langle x_1, x_2 \rangle = \langle 1, 0 \rangle$
 or
 $\langle 1, 1 \rangle$
 as first candidates

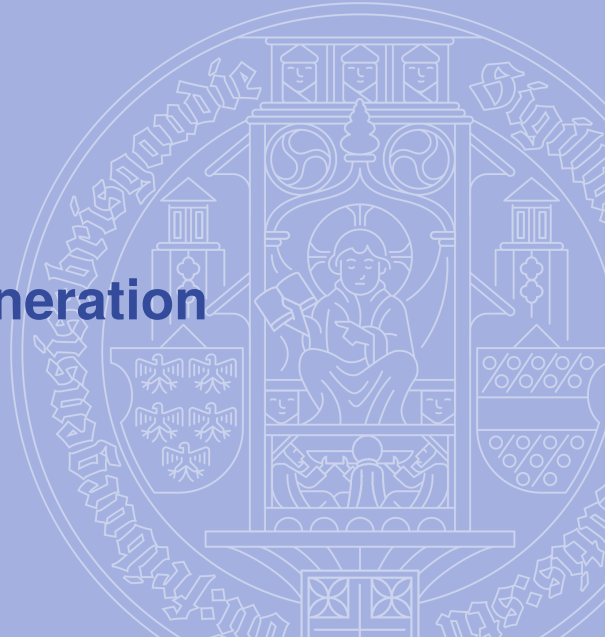
Detecting a Stuck-Open Fault

- $x_1 = 1, x_2 = 0$:
 - in the fault free circuit $z = 0$.
 - in the faulty circuit:
 - Q_3 does not conduct. z cannot go to 1.
 - Q_2 does not conduct. Q_1 is disconnected. z cannot go to 0.
 - z retains its previous value.
- To detect the fault, make sure the previous value of z is 1.
 - achieved by $x_1 = 0, x_2 = 0$.
 - a test: $\langle 00, 10 \rangle \Rightarrow z = \langle 1, 0/\mathbf{1} \rangle$

Summary

- Fault models are used to **model the effects** of physical defects.
- They provide a target for test generation.
- Different fault models will result in different sets of tests that will detect different types of defects.
- The selection of the model will **depend on the time** that can be invested in test generation and the **number of tests** that may be used.

Introduction: Test Generation



A Test Generation Process

- **Generate** tests at the **behavioral** level.
- **Simulate** the tests at the **functional** level.
 - Generate additional tests for the remaining faults.
- **Simulate** at the **structural** level.
 - Generate additional tests for the remaining faults.
- **Simulate** at the **switch** level ...

Commonly Used Process

- **Higher levels** are considered by the **designer**, who provides a 'functional' test sequence.
- **Test generation** considers the structural level and possibly the layout for realistic faults.
- Generates tests for **(close to) 100%** of the **single stuck-at faults**.
- Augment the test set by tests for:
 - **delay faults** (transition faults and/or a subset of path delay faults).
 - I_{DDQ} tests for bridging faults.

Complexity

- Test generation for a single stuck-at fault is an NP-complete problem.
- A complete test generation procedure has complexity which is (for the moment) exponential in the number of inputs.
- In practice, various optimization techniques and heuristics exist to solve the problem efficiently.

Circuits

- Actual **manufactured circuits** that have to be tested (they may be fault free or faulty). We have access only to their inputs and outputs.
- **Models** (computer representations) for which internal details are known.
 - **fault free circuit**.
 - **faulty circuits** with modeled faults injected into them.
 - used for test generation, fault simulation, etc.

Fault Detection and Redundancy

Combinational Circuits

- Combinational circuit N with output function $Z(x)$ (input variables x).
- A specific **input vector** is denoted by t . The response of N to t is $Z(t)$ (also a vector if N is a multiple output circuit).
- The presence of a **fault** f **transforms** N into a **new circuit** N_f . N_f is also combinational, and has an output function $Z_f(x)$.

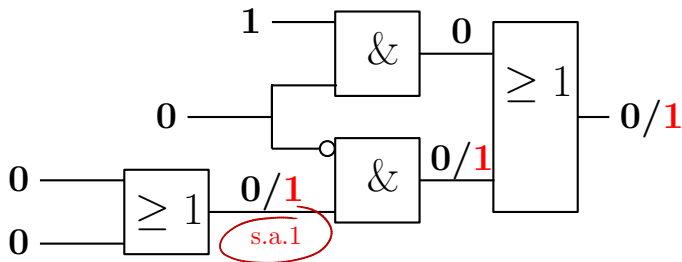
Fault Detection

Definition

A test vector t **detects** a fault f iff $Z_f(t) \neq Z(t)$.

- For a set of faults $F = \{f_1, f_2, \dots, f_n\}$, it is necessary to use a test set $T = \{t_1, t_2, \dots, t_m\}$.
- A test t for a fault f satisfies $Z(t) \oplus Z_f(t) = 1$. It is possible to find a test t (or all the tests for f) by solving this equation.
- For a multi-output circuit:
 - $Z(t) = z_1(t)z_2(t) \dots z_m(t)$
 - $Z_f(t) = z_{1f}(t)z_{2f}(t) \dots z_{mf}(t)$
 - $Z(t) \oplus Z_f(t) = (z_1(t) \oplus z_{1f}(t)) + (z_2(t) \oplus z_{2f}(t)) + \dots + (z_m(t) \oplus z_{mf}(t))$

Sensitization



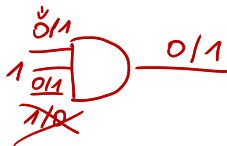
The test **activates** the fault, **propagates** the fault to a primary output, and **sensitizes** several lines (including the output) to the fault.

Terms

- The test **activates** the fault—assigns to the fault site different fault free and faulty values.
- The test **propagates** the fault to a primary output
 - causes all the lines along at least one path between the fault site and the output to have different fault free and faulty values.
 - causes an output to have different fault free and faulty values.
- A line whose value under t is different in the faulty and fault free circuits is said to be **sensitized** to the fault f by the test t .
- A path composed of sensitized lines is called a **sensitized path**.

Sensitization

AND, OR, ~~NAND~~, NOR, inverters
 $i = 0$ $\bar{c} = 1$



Lemma 1

Let G be a gate with **inversion** i and **controlling value** c . Suppose that the output of G is sensitized to a fault f by a test t .

1. All the inputs of G sensitized to f have the same value, a , in the fault free circuit.
2. All the inputs of G not sensitized to f have value $\neg c$.
3. The output of G has value $a \oplus i$ in the fault free circuit.

Proof.

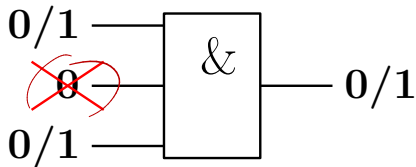
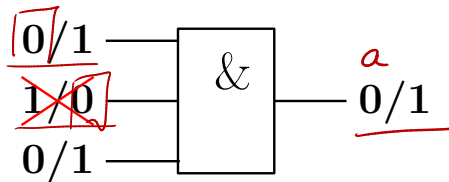
Without proof. □

Terms

- **Inversion i :**
 - AND, OR, buffer: $i = 0$
 - NAND, NOR, inverter: $i = 1$
- **Controlling value c :** If the value of a single input of a gate is c , the output value is determined independent of other input values.
 - AND, NAND: $c = 0$
 - OR, NOR: $c = 1$

Examples of Lemma 1

$$a = 0$$



Sensitization



$p=0 \rightarrow$ even number of inversions

$p=1 \rightarrow$ odd of "inv."

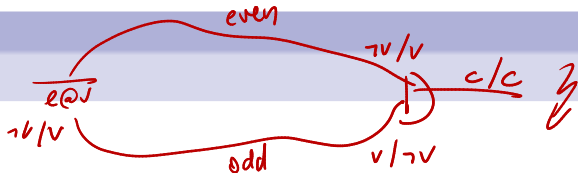
Lemma 2

Let j be a line sensitized to the fault l s.a. v by a test t , and let p be the inversion parity of a sensitized path between l and j .

1. The value of j under t in the fault free circuit is $\neg v \oplus p$.
2. if there are several sensitized paths between l and j , then all of them have the same inversion parity.

Proof.

Without proof.

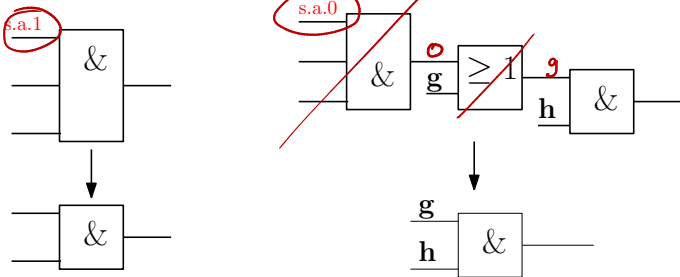


Detectability

- A fault f is said to be **detectable** if there exists a test t that detects it.
- Otherwise, f is an **undetectable** fault. $Z_f(x) = Z(x)$.
- The presence of an undetectable fault may invalidate a test for a detectable fault.

Redundancy

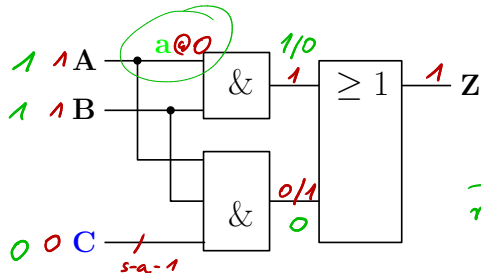
- A combinational circuit that contains an **undetectable stuck-at** fault is said to be **redundant**.
- A **redundant circuit** can be **simplified** by removing at least one line.



Properties of Redundant Faults

1. After removing a redundant fault f , a detectable fault g may become redundant.
2. After removing a redundant fault f , a redundant fault g may become detectable.
3. Two redundant faults f and g may be detectable if present simultaneously.

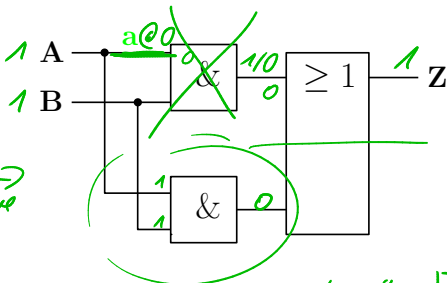
Properties of Redundant Faults—Property 1.)



C stuck at 1 (C-s-a-1) redundant
(A and B must be 1, propagation blocked at OR gate)

a-s-a-0 detectable ($ABC = 110$,
 $Z = 1/0$)

remove C

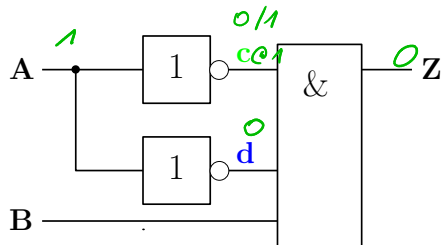


C removed

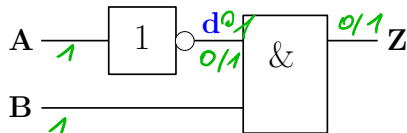
$\hookrightarrow \begin{matrix} a \\ b \end{matrix} = D$

a-s-a-0 redundant ($A = 1$ for
activation, $B = 1$ for propagation
through AND gate, propagation
blocked at OR gate)

Properties of Redundant Faults—Property 2.)



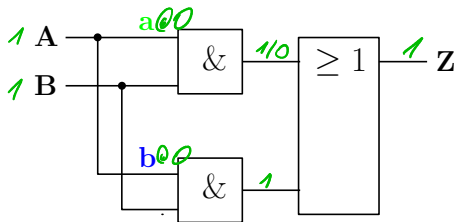
c-s-a-1 and **d-s-a-1** redundant (A must be 1 for activation, hence both c and d are 0, propagation blocked)



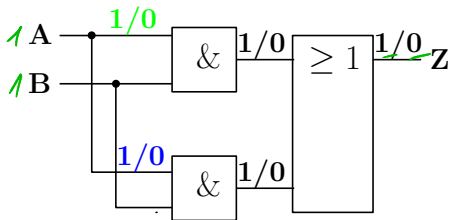
c removed

d-s-a-1 detectable (via $A = 1$ and $B = 1$)

Properties of Redundant Faults—Property 3.)

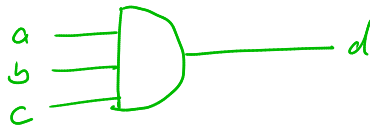


Single stuck-at faults **a-s-a-0**,
b-s-a-0 redundant (A and B must
be 1, propagation blocked at OR
gate)



Double stuck-at fault **a-s-a-0** and
b-s-a-0 detected by $A = 1$ and
 $B = 1$

Fault Collapsing — Equivalent Faults



- Two faults f and g are **functionally equivalent** if and only if $Z_f(x) = Z_g(x)$.
- For an n -input AND gate:
 - We can define $2(n+1)$ single stuck-at faults.
 - The **s.a.0** faults on the **inputs** and on the **output** are **equivalent**.
 - It is sufficient to consider the **output s.a.0** fault and **all the s.a.1** faults for a total of $n+2$ faults.
- Analogous for other gates ...

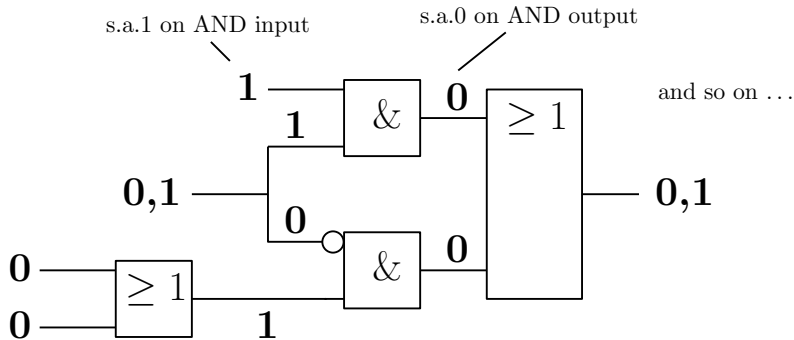
a	b	c	d			
0	0	0	0			
0	0	1	0			
0	1	0	0			
0	1	1	0			
1	0	0	0			
1	0	1	0			
1	1	0	0			
1	1	1	0			

a	b	c	d
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

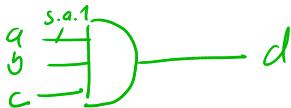
$w@0 \equiv b@0$
 $\equiv c@0$
 $\equiv d@0$

Fault Collapsing — Equivalent Faults

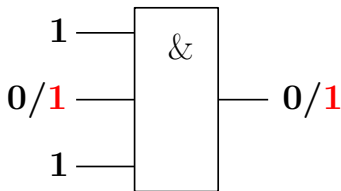
- For every n -input gate, use $n + 2$ non-equivalent faults.
- Reduces the number of single stuck-at faults that need to be considered.



Fault Collapsing — Fault Dominance



- Let T_g be the set of all the tests that detect a fault g . We say that a fault f **dominates** a fault g if every test that detects g also detects f , or $T_g \subseteq T_f$.
- The **s.a.1** fault on the **output of an AND** gate **dominates** the **s.a.1** fault on every **one of its inputs**.



$$T_{a@1} \subseteq T_{d@1}$$
$$T_{b@1} \subseteq T_{d@1}$$

$$T_{c@1} \subseteq T_{d@1}$$

$$T_{a@1} = \{011\}$$

$$T_{b@1} = \{101\}$$

$$T_{c@1} = \{110\}$$

$$T_{d@1} = \{000, 001, 010, 100, 110, 101, 011\}$$

Fault Collapsing — Fault Dominance

- **Remove** the **dominating output fault** of every gate from the set of faults for test generation.
- **Not used in practice.** If the input faults are undetectable but the output fault is detectable, the output fault may remain undetected if removed, even though it is detectable.

Fault Collapsing — Fault Dominance

Different Definition

- We say that a fault f **diagnostically dominates** a fault g if for every test t and every output z of the circuit, if g is detected on z then f is also detected on z .

Example:

	Outputs $z_1 z_2 z_3$	
test	fault f	fault g
00	001	000
01	111	101
10	100	100
11	000	000

0 means 'fault is not detected'
1 means 'fault is detected'